Assignment 6: Implementation of Data Integrity Constraints

Database: MySQL | Project: Airline Reservation System

1. Objective

Implement and verify primary keys, foreign keys, UNIQUE, CHECK and DEFAULT constraints.

2. Constraint purposes

Constraint	Example	Purpose
PRIMARY KEY	passenger_id	Uniquely identifies a passenger record
FOREIGN KEY	Bookings.flight_id	Prevents bookings for nonexistent flights
UNIQUE	Passengers.email	Prevents duplicate email addresses
NOT NULL	Flights.flight_number	Requires an essential value
CHECK	Aircraft.capacity > 0	Restricts invalid capacity values
DEFAULT	Payments.payment_status DEFAULT 'Pending'	Provides a value if omitted
UNIQUE + FK	Payments.booking_id	Supports one payment per booking

3. Integrity tests

Run the tests after loading valid data from Assignment 7. Each invalid test is expected to fail; do not treat expected failures as successful insertions.

Test 1 — duplicate primary key

```
INSERT INTO Passengers
VALUES (101,'Duplicate Passenger','dup@example.com','9000000000','P99999');
```

Expected: duplicate primary-key error because passenger ID 101 already exists.

Test 2 — duplicate email

```
INSERT INTO Passengers
VALUES (199,'Test Passenger','arjun.sharma@example.com','9000000001','P19999');
```

Expected: unique-constraint error because the email already exists.

Test 3 — invalid foreign key

```
INSERT INTO Bookings
VALUES (199,'2026-10-23','20A','Economy',999,101);
```

Expected: foreign-key error because passenger 999 does not exist.

Test 4 — invalid CHECK value

```
INSERT INTO Aircraft
VALUES (199,'Test Aircraft',-10,'Test Airline');
```

Expected: CHECK constraint error because capacity must be greater than zero.

Test 5 — DEFAULT value

```
INSERT INTO Payments
(payment_id,payment_date,amount,payment_method,booking_id)
VALUES (199,'2026-10-23',5000,'UPI',199);

SELECT payment_id,payment_status
FROM Payments WHERE payment_id=199;

DELETE FROM Payments WHERE payment_id=199;
```

Expected output: payment_status = Pending because the DEFAULT value is applied when the status is omitted.

Test 6 — duplicate booking payment

```
INSERT INTO Payments
VALUES (200,'2026-10-23',6000,'Card','Paid',101);
```

Expected: duplicate-key error if booking 101 already has a payment.

4. Verify relationships

```
SELECT b.booking_id, p.full_name,
       f.flight_number, pay.amount, pay.payment_status
FROM Bookings b
JOIN Passengers p ON b.passenger_id = p.passenger_id
JOIN Flights f ON b.flight_id = f.flight_id
LEFT JOIN Payments pay ON b.booking_id = pay.booking_id;
```

The query returns booking records joined to matching passengers, flights and payments. Foreign keys enforce the validity of the related references.

5. Conclusion

Constraints prevent duplicates, invalid references and inappropriate values. Execute the tests in MySQL and capture screenshots of the actual output and error messages for submission.